

JYP엔터테인먼트 IT LAB · Software Engineer / Frontend (경력) 지원

프로토타입은 빠르게, 완성도는 끝까지.

산출물이 없는 자리에서 시작한 일이 많았습니다. 빨리 만들어 확인하되, 끝은 흐리지 않는 쪽으로 일해왔습니다. 미국 클라이언트 사이트를 **2년 5개월** 원격 단독으로 맡아 계약 종료 시점에 발행을 자동화해 인계했고, Digital SAT는 화면 정의가 없는 상태에서 IA · 와이어프레임 · Figma를 직접 만들어 합의한 뒤 React로 구현했습니다.

askedtech.com

[Next.js](#) · TS · 재직 중

precisionrct.com

[2년 5개월](#) · 미국 D.C.

mstone-demo.vercel.app

[Next.js](#) App Router · 정적 생성

[Apple Pop](#)

[사과게임](#) · iOS · Android 출시

409 → 4

GraphQL 중첩 조회 쿼리 수. DataLoader 로 배치해 99% 감소 — 스키마부터 직접 구현하고 계측했습니다

기획 → 운영

미국 워싱턴 D.C. 의료기관 사이트 **한 건**을 기획 · IA · 디자인 · 개발 · SEO · 운영까지 단독으로 맡은 2년 5개월. 100% 원격 · 영어 서면

서버 60ms

지금 맡은 서비스. 느린 원인이 서버가 아니라는 것부터 확인하고, 병목을 전달 계층으로 좁혀 2MB짜리 폰트 한 개를 찾았습니다

경력	AIS 2021 · 엔텔스 2021.12–2022.09 · 교육공간 2023.06–2024.01 · Precision Micro Endodontics 2024.01–2026.05(프리랜서) · 러닝스파크 2026.06–2026.08
담당 범위	요구사항 협의 · IA · 화면 설계 → React / Next.js 구현 → API 응답 계약 정의 → 배포 · 운영
학력	한국방송통신대학교 컴퓨터과학과 2024.03–2027.03
FRONTEND	React · Next.js · TypeScript · JavaScript(ES6+) · HTML / CSS
MOBILE	React Native · Expo · EAS CI/CD
API · 국제화	REST / RPC · Supabase Edge Functions · GraphQL(구현 · 저장소 공개) · i18next
품질 · 운영	Lighthouse · Web Vitals · 접근성 · 반응형 · Sentry · 무중단 배포
그 외	Node.js / NestJS · Java / Spring · PHP · MariaDB · MongoDB

역할이 다른 사용자들이 한 업무 시스템을 함께 쓰는 환경에서 화면을 어디까지 공유하고 어디부터 갈라야 하는지 정해왔습니다. 멘토와 학교, 학원과 강사와 학생, 사용자와 운영자가 각각 자기에게 필요한 범위만 보도록 갈랐습니다. 지금 맡은 일에서는 자동화를 어디까지 기계에 맡기고 어디를 사람에게 남길지 정하고 있습니다 — 수집 · 분류 · 요약은 자동이고 발행은 사람이 누릅니다. 내부 구성원과 아티스트, 각 사업 조직이 더 효율적으로 일하도록 돕는 시스템이라면, 권한과 자동화의 경계를 어디에 두느냐가 화면 구조를 정하고 쓰는 경험을 가른다고 봅니다.

01. 주요 프로젝트

2026.06 - 2026.08 · 계약직 · IT부서

askedtech · 러닝스파크 재직 중

Next.js TypeScript NestJS MongoDB AWS EC2

에듀테크 뉴스 · 커뮤니티 서비스. 사용자 웹 · 관리자 CMS · API 세 서비스의 화면과 응답 계약을 맡고 있습니다.

문제

자료는 RSS와 메일함으로 나뉘어 들어오고, 발행까지 섹션 분류 · 요약 · 검수를 거쳐야 합니다.

설계

- 수집 · 분류 · 요약은 기계가 하고, 발행 버튼은 사람이 누릅니다. 승인이 번거로우면 사람들은 파이프라인을 우회해 손으로 보내기 때문에, 통제를 조이는 대신 한 번에 훑고 끝나게 만들었습니다.

결정

- 프롬프트보다 먼저 정한 건 “LLM 없이도 전체를 돌려볼 수 있는가”였습니다. 호출부를 주입 가능한 클라이언트로 분리하고 드라이런 모드를 뒀습니다.
- API는 모듈별로 생성 · 조회 · 수정을 다른 DTO로 나눴고(28개 중 21개), 두 프론트가 거기서 내린 타입 한 벌을 공유합니다. 한쪽 화면 요구로 응답 모양을 바꾸면 다른 쪽이 조용히 깨지기 때문입니다.
- SNS 자동 발행은 만들지 않았습니다. 손익분기가 **53주**로 나와, 사람이 한 번 누르는 반자동 MVP를 제안해 채택됐습니다.



자동으로 흘러가다가 승인 칸에서 한 번 멈춥니다. 사람이 누르지 않으면 아무것도 나가지 않습니다.

EC2 3대 무중단 배포 · 사이트맵 자동화 7,000+ URL

인계 전 실측 - 서버 응답이 60ms인데 화면이 느려, 병목을 전달 계층으로 좁혔습니다. 전송량 18MB 가운데 폰트 한 개가 2MB였습니다.

askedtech.com

precisionrct.com

PHP JSON jQuery Bootstrap 5 SiteGround

미국 워싱턴 D.C. 근관치료 전문 치과의 사이트. 기획 · IA · 디자인 · 개발 · SEO · 운영까지 2년 5개월간 100% 원격으로 맡았고, 현지 담당자와 요구사항 합의부터 인수인계까지 영어 서면으로 진행했습니다.

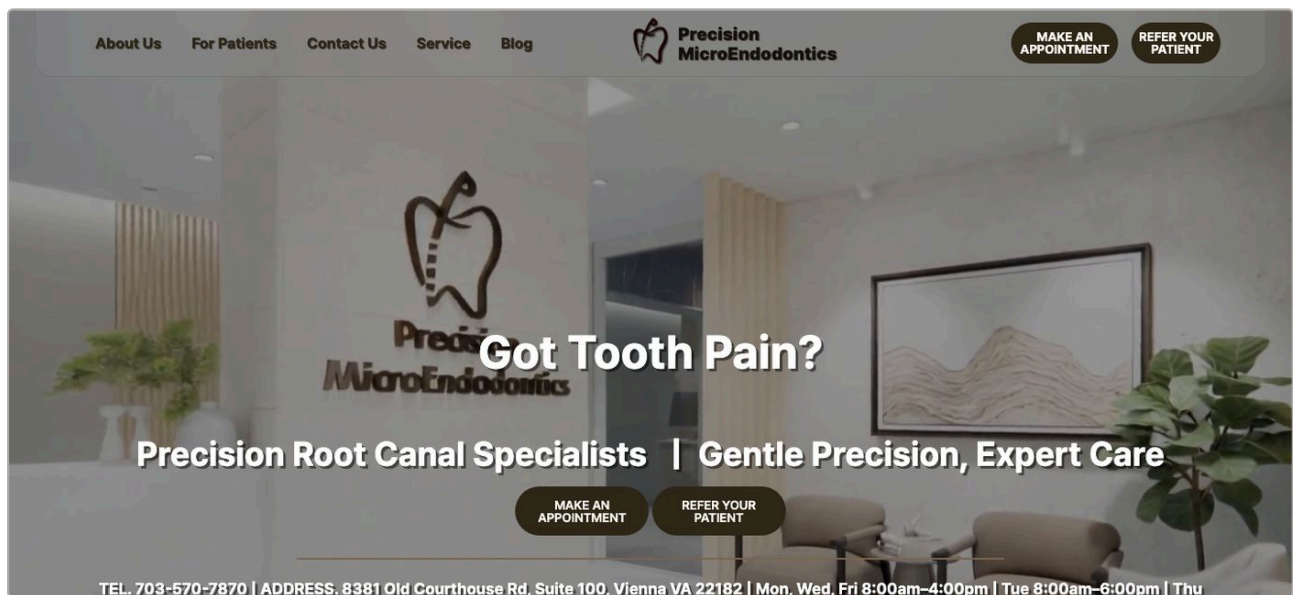
설계

CMS 없이 PHP + JSON 파일 기반 렌더링 구조를 직접 설계했습니다. WordPress를 었었다면 플러그인과 보안 패치를 원격 1인이 2년 내내 따라가야 했습니다.

결정

계약 종료 시점에 발행을 자동화해 인계했습니다. 제가 빠진 뒤에도 담당자가 코드를 건드리지 않고 새 글을 올릴 수 있어야 사이트가 계속 굴러갑니다.

반응형 / 크로스브라우징 · 메타데이터 · 사이트맵 · OG · 호스팅 · SSL · 백업 · 장애 대응까지 단독



환자와 의뢰 치과 의사 같은 첫 화면에 들어옵니다. 두 사이트로 나누지 않고 버튼 두 개로 갈랐고, 전화번호와 진료시간은 스크롤 없이 보이는 자리에 뒀습니다 — 문의의 대부분이 그 둘이라, 보여주고 싶은 것보다 찾으려 온 것을 먼저 올렸습니다.

precisionrct.com

Digital SAT 웹앱 · 교육공간

React Java MariaDB Figma

학원에 공급하고 학생이 쓰는 SAT 모의고사 운영 웹앱 — 자사 학원에서도 같은 제품을 썼습니다. 학원 · 강사 · 학생 세 주체가 같은 응시 데이터를 서로 다른 범위로 봅니다. IA · 화면 설계부터 React 구현까지 맡았습니다.

문제

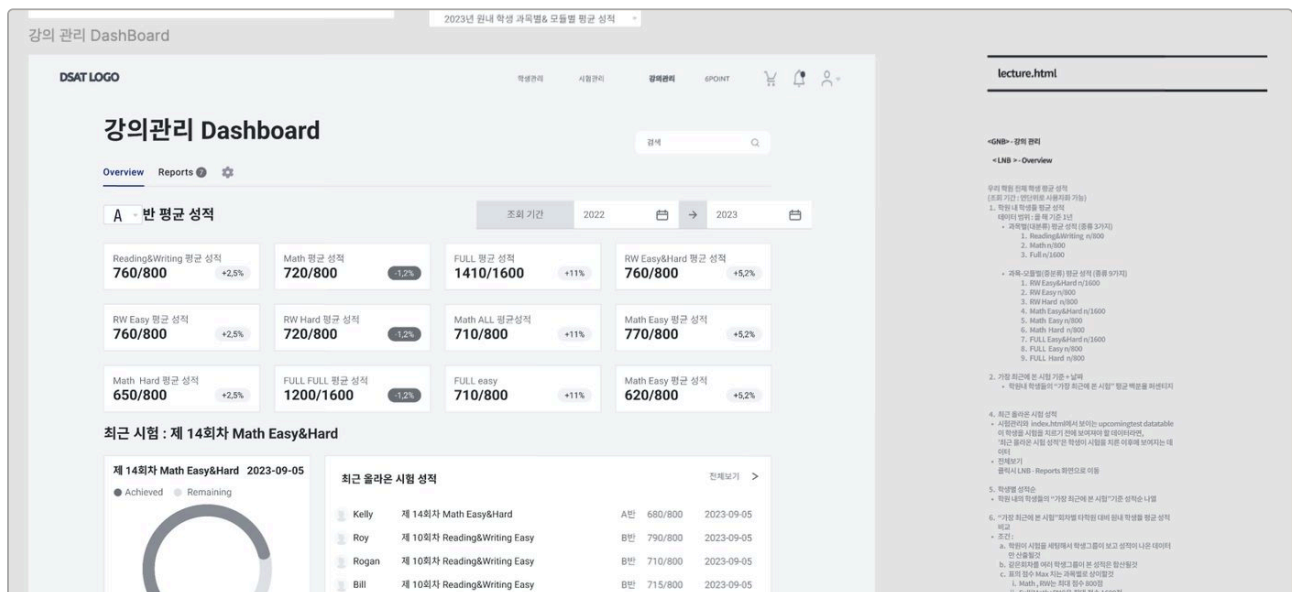
학생 · 시험 · 강의 · 포인트 네 모듈을 만들어야 하는데 화면 정의가 없었습니다.

설계

IA와 와이어프레임을 먼저 만들고 Figma 프로토타입으로 합의한 뒤 React로 구현했습니다. 요구사항을 기다리지 않고, 합의부터 시작했습니다.

결정

- 응시 상태 자동저장 · 복원은 실패 시나리오에서 출발했습니다. 시험 중 브라우저가 닫히면 답안이 사라지므로 저장 책임을 학생이 아니라 시스템으로 옮겼습니다.



화면 정의가 없어, 쓸 사람들을 직접 만나 물어가며 정한 산출물입니다. 왼쪽이 스토리보드, 오른쪽이 각 영역의 데이터 범위와 계산 규칙을 적은 기능정의서입니다. 학원 · 강사 · 학생이 같은 성적 데이터를 서로 다른 범위로 보기 때문에, 화면을 그리기 전에 어느 지표를 누구에게 어디까지 보여줄지부터 정하고 그것을 한 장에서 합의했습니다.

IA · 와이어프레임 · 프로토타입 · UX/UI를 직접 만들고 구현까지 담당

디자인 시스템 · 다국어

Apple Pop 사과게임

색 · 타이포 · 라운드 · 간격 · 모션을 **토큰으로 먼저 정하고 그 위에 재사용 컴포넌트를 얹었습니다**. 22개 로케일을 붙이면 번역 파일보다 레이아웃이 먼저 깨집니다 — 같은 버튼이 언어에 따라 두 배로 길어지므로, 문자열을 전부 분리하고 폰트 폴백 순서를 정한 다음에 번역을 넣었습니다.

React Native · TypeScript · Supabase Edge Functions(REST / RPC) · i18next 22개 로케일 · EAS CI/CD · Sentry · iOS · Android 양대 마켓 출시

[App Store](#) [Google Play](#)

02. 성능 · 접근성 실측

제가 만든 결과물을 같은 조건으로 재고, 편차마다 원인을 붙였습니다.

측정: Chrome Lighthouse (모바일 / 데스크톱 프로파일), 시크릿 창 · 캐시 비활성 · 2026년 8월 측정. — 는 측정하지 않은 항목입니다.

대상	환경	성능	접근성	모범사례	SEO	LCP · CLS
mStone 개인 · Next.js	데스크톱	100	100	100	100	0.8s · 0.003
정적 생성						
처음부터 Next.js로 만든 경우.						
mStone 개인 · Next.js	모바일	76	100	100	100	4.7s · —
정적 생성						
반복 측정하면 76에서 96까지 흔들립니다. 회선이 아니라 모바일 CPU 스로틀링에 민감하다는 뜻입니다.						
precisionrct.com	모바일	60	80	100	92	11.7s · 0.005
jQuery · Bootstrap 5 기반 자산을 2년 5개월 운영한 사이트. 계약 종료 시점의 1순위는 발행 자동화 인계였고 성능은 뒤로 미뤘습니다. 미룬 것도 판단입니다.						
jaykim-v.github.io/fe	데스크톱	100	100	100	100	0.5s · 0
jaykim-v.github.io/fe	모바일	100	100	100	100	1.4s · 0

위 진단을 그대로 적용한 결과입니다. 스크립트는 화면 인터랙션용 **gzip 2.5KB** 한 벌뿐이고 직접 짰습니다 — 같은 동작을 GSAP · Lenis로 붙이면 gzip 35KB에 외부 도메인 핸드셰이크가 세 번 더 붙습니다. 웹폰트는 self-host 서브셋 4파일 104KB. CLS 0은 본문 도판마다 width · height를 지정해 자리를 미리 잡은 결과입니다. 남은 미세한 밀림은 폴백 서체의 ascent · descent를 Pretendard 실측값에 맞춰 없었습니다.

정적 문서라 100 자체가 어려운 일은 아닙니다. 여기서 보실 것은 점수가 아니라
편차이고, 편차마다 원인을 적어줬다는 점입니다.

이 표를 만든 페이지입니다. 대비비 계산표 · 토큰 규칙 · 개인 프로젝트 상세는 jaykim-v.github.io/fe 에 있습니다.

03. 일하는 방식

같은 백엔드 위에서 화면을 가릅니다

역할이 다른 사용자들이 한 시스템을 함께 쓰는 구조를 계속 다뤄왔습니다 — 엔텔스 · 교육부
원격영상 진로멘토링(멘토 / 학교) · Digital SAT(학원 / 강사 / 학생) · precisionrct(환자 /
의뢰 치과의) · askedtech(사용자 / 운영자) · 할리스 리뉴얼(고객 / 창업 예정자). 그중
진로멘토링은 멘토 포털과 학교 포털이 같은 백엔드를 쓰는 구조라, 백엔드 담당과 응답
계약을 먼저 맞춘 뒤 화면과 권한만 갈랐습니다.

Digital SAT에서는 화면 정의가 없어 IA · 와이어프레임 · Figma를 직접 만들고, 쓸 사람들과
마주 앉아 어느 지표를 누구에게 어디까지 보여줄지 합의했습니다. 선은 혼자 그을 수
없습니다 — 백엔드와는 응답 계약으로, 쓰는 쪽과는 화면으로 맞춰야 같은 선이 됩니다.
할리스 리뉴얼에서는 팀 리드로 고객과 창업 예정자의 동선을 갈랐습니다.

어려운 건 화면을 두 벌 만드는 일이 아니라 어디까지 공유하고 어디부터 갈라야 하는지 선을
긋는 일이었습니다. 화면을 나누는 기준이 곧 권한 모델이고, 권한 모델은 나중에 붙일 수
없습니다.

04. 핵심 역량과 근거

IT LAB이 맡길 일에 바로 쓰이는 역량을, 실제로 운용한 서비스에서 뽑아
정리했습니다.

역량마다 그 일을 실제로 한 서비스와, 그 서비스에서 무엇을 했는지를 적었습니다.

역량	근거
React · Next.js	askedtech 사용자 웹 · 관리자 CMS 두 Next.js 앱을 동시에 운용 · Digital SAT를 React로 전면 구현
TypeScript	askedtech 전 구간. 두 프론트가 API 응답 타입을 각자 선언하지 않고 DTO에서 내린 한 벌을 공유합니다
디자인 · 기획 의도의 기술 구현	엔텔스에서는 발주처가 정한 화면 정의를 받아 구현했고, Digital SAT에서는 산출물이 없어 IA · 와이어프레임 · Figma를 직접 만들어 합의한 뒤 구현했습니다 — 받는 쪽과 만드는 쪽을 모두 했습니다

웹 성능 · 접근성	askedtech 인계 전 측정으로 느린 원인이 서버가 아님을 확인하고 전달 계층으로 좁혔고, precisionrct에서는 2년 5개월간 재면서 무엇을 먼저 고치고 무엇을 미룰지 정했습니다. 접근성은 대비비 전 조합 계산 · 미달 조합 사용 규칙 · <code>:focus-visible</code> · <code>scope</code> 구현(웹 상세 — jaykim-v.github.io/fe)
REST API 연동	askedtech에서 세 서비스를 잇는 응답 계약을 정하고 운용 · Apple Pop은 Supabase Edge Functions로 REST / RPC 작성
GraphQL	개인 검증 — 스키마 · 리졸버 · DataLoader를 직접 구현해 중첩 조회 409쿼리를 4쿼리로 줄인 것을 계측 — github.com/JayKim-v/graphql-n-plus-one
협업 주도	엔텔스에서 두 포털이 같은 백엔드를 쓰는 구조라 백엔드 담당과 응답 계약을 조율 · SNS 자동 발행의 손익분기를 53주로 계산해 반자동 MVP를 제안하고 채택까지 이끌었습니다
글로벌 유저 경험	미국 클라이언트 사이트를 2년 5개월 원격 단독 운영 — 합의부터 인계까지 영어 서면 · Apple Pop 22개 로케일, 문자열 외부화 후 폰트 폴백 순서를 정하고 번역 적용

“열심히 노력하다가 갑자기 나태해지고,
잘 참다가 조금해지고,
희망에 부풀었다가 절망에 빠지는 일을 또다시 반복하고 있다.
그래도 계속해서 노력한다면 수채화를 더 잘 이해할 수 있겠지.
그게 쉬운 일이었다면, 그 속에서 아무런 즐거움도 얻을 수 없었을 것이다.
그러니 계속해서 그림을 그려야겠다.”

빈센트 반 고흐가 동생 테오에게

부풀던 날에도, 무너지던 날에도,
오래 곁에 두고 꺼내 보는 문장입니다.

완벽하지 않은 그 날이 시작만 하는 사람과 끝까지 완주하는 사람을
결정짓는 날이다.

완주하는 개발자, 김재희입니다.